

A Case Study on the Impact of Customer Communication on Defects in Agile Software Development.

Mikko Korkala¹, Pekka Abrahamsson² and Pekka Kyllönen²

¹Department of Information Processing Science
P.O.Box 3000, FIN-90014 University of Oulu, Finland

²VTT Technical Research Centre of Finland
P.O.Box 1100, FIN-90571, Oulu, Finland

Mikko.Korkala@oulu.fi; Pekka.Abrahamsson@vtt.fi; pekka.kyllonen@vtt.fi

Abstract

Effective communication and feedback are crucial in agile development. Extreme Programming (XP) embraces both communication and feedback as interdependent process values which are essential for projects to achieve successful results. Our research presents the empirical results from four different case studies. Three case studies had partially onsite customers and one had an onsite customer. The case studies used face-to-face communication to different extents along with email and telephone to manage customer-developer communication inside the development iterations. Our results indicate that an increased reliance on less informative communication channels results in higher defect rates. These results suggest that the selection of communication methods, to be used inside development iterations, should be a factor of considerable importance to agile organizations working with partially available customers. This paper also proposes some guidelines for selecting proper communication methods.

1. Introduction

Misunderstood or erroneous requirements are often the reason why software systems are late and unreliable [1]. Literature and practice show that agile methods work with unstable requirements which are managed through short development cycles and efficient communication (e.g. [2, 3]). Agile development approaches tend to rely on effective personal communication and collaboration between the project participants [2, 4].

In fact, two agile principles (<http://agilemanifesto.org/principles.html>) directly

emphasize the importance of customer-developer communication. Face-to-face communication is identified as the most efficient means of communication between the participants. Moreover, the daily collaborative work of business people and developers demands efficient verbal communication between the customer and the developers. In addition to communication, feedback is also crucial in agile development. The most popular agile method Extreme Programming (XP) embraces both communication and feedback as interdependent process values. Previously, XP went to extremes with customer communication and feedback by proposing an *Onsite Customer*, a real person constantly available for information [3]. Although some authors (e.g. [5-8]), have found weaknesses in this practice, on the positive side, having an onsite customer when possible enables efficient communication and feedback since the customer is always and easily available for information. Even though the practice has its advantages, it is not always possible to have the customer constantly onsite due to various reasons (e.g. the customers are too valuable or too remote to be onsite) [9]. The *Onsite Customer* has now been replaced with the *Real Customer Involvement* practice which does not assume fulltime customer presence, but suggests that the customer can participate in weekly and quarterly meetings [10].

As demonstrated above, the importance of communication has been shown to be paramount in agile development. Less however is known about the implications of communication and feedback in this context. The purpose of this paper is therefore twofold. First, this paper explores the impact of customer communication and feedback on software defect rates. Second, it briefly discusses the selection

of customer communication channels based on the existing literature.

The empirical material is based on four case studies. From one of the case studies, the researchers were able to capture reliable metrics about the defect division and the times required to fix those defects. Defect follow-up provides a means for increasing our understanding of how well a team has understood the customer's intentions. The other three case studies, including one onsite customer project, are used to show trends in the development of defect densities. These cases applied different communication channels, varying in their richness. The richness of the communication channels is observed from the viewpoint of the Media Richness Theory (MRT) [11, 12], and personal suggestions on the information richness of different communication mediums by Ambler [13] and Cockburn [4].

We claim that effective and rich mid-iteration communication¹ is a highly important factor in cases where rich customer communication is not always available, and the lack of high-bandwidth (e.g. face-to-face) communication and an increasing reliance on leaner communication channels results in higher defect rates, despite careful face-to-face requirements analysis.

The principal findings of the study are as follows: It was found that an average of 25.3% of all tasks involved fixing defects in the three non-onsite customer projects. The case proving accurate metrics about the defects indicated that an average of 62.6% of all defects could have been avoided by either having an onsite customer or a more effective mid-iteration communication mechanism. These results indicate that the selection of a mid-iteration communication medium, especially, must be given significant attention in agile development efforts with partially available customers. Some suggestions to overcome these challenges will be presented in this paper.

This paper is organized as follows: section 2 discusses the relevant communication literature, focusing on the MRT and communication in agile context; Section 3 explains the research settings; the empirical research results and our discussion on agile communication are in section 4; and the conclusions are drawn in section 5.

¹ Mid-iteration communication is used for the customer-developer communication taking place inside any particular iteration.

2. Related literature

Effective communication is an essential success factor for any software development effort (e.g. [3, 14, 15], but according to Cockburn [4] it is far from being an easy task. Sometimes the idea to be expressed is not even fully conscious, and for the communication to be effective the participants must have a shared experience of the subject that is being discussed. This shared experience is needed for the participants to compensate for the gaps in communications. [4]

Section 2.1 discusses the Media Richness Theory which, despite criticism, is possibly the most well-known theory of media selection. Section 2.2 focuses on communication in the context of agile software development.

2.1. Media richness theory (MRT)

Media Richness Theory (MRT) [11, 12] is perhaps the most prominent theory of media selection, even though it has been widely criticized (e.g. [16-18]). The theory suggests that matching the task needs to mediums ability to convey information will improve task performance and that certain media are better capable of transmitting uncertain or equivocal information [11, 12].

Uncertainty is interpreted as the lack of information [19], while equivocality means that there are multiple, possibly conflicting interpretations for the information [16]. The values of these variables can be classified as high or low [11]. According to Daft, Lengel and Trevino, uncertainty leads to data acquisition whereas equivocality results in the exchange of subjective views, definition of the problem and resolution of disagreements [12]. Daft and Lengel [11] suggest that equivocal tasks should be managed through rich communication channels, while leaner channels are effective for processing standard data and well understood messages.

The richness of different communication mediums is based on a blend of four criteria [12]. The characterization of these variables is complemented by the perceptions of online media by Graveline, Geisler and Danchak [20], as follows:

1) Feedback, instant feedback enables asking questions and making corrections. The capability of feedback relates to synchronicity of feedback; online media are either synchronous or asynchronous [20];

2) **Multiple cues**, different cues (e.g. body language, voice inflection) can be a part of the message. Some online media are capable of transmitting multiple cues (e.g. videoconferencing), some are primarily single channelled (e.g. email) [20];

3) **Language variety**, refers to the use of different languages, such as numbers which convey precision and natural language conveying the understanding of a broader set of ideas. Written or typed language provides more precise language [20];

4) **Personal focus**, A message will be conveyed more fully when personal feelings are infusing the communication. However, sometimes lesser emotional content can be better. Cockburn [4] provides an example of a project leader and her team who used telephones because she was too aggressive with her emotions in person.

MRT suggest that the richest communication channel enables instant feedback, is capable of transmitting multiple cues, has a high language variety and it can transmit emotional contents.

Based on [12], the following summary of the richness of different communication channels can be made. Face-to-face is the richest form of communication since it enables instant feedback and it is able to transmit multiple cues [12]. Videoconferencing is capable of sending both video and audio, and it enables fast feedback [12]. Videoconferencing is however a somewhat leaner medium than face-to-face and it restricts some visual cues, but it has more information capacity than telephone [12]. Email has capabilities similar to telephone, but the cues are filtered out [12].

As mentioned above, MRT has been criticized by several authors. For example, Dennis and Valacich [16] conclude that the richest medium is defined by the situation in which it is used. The richness of medium depends on its ability to provide the capabilities needed by the situation. These capabilities include the individuals, tasks and the social context in which they interact [16]. They also state that media is not monolithic, since media are capable of possessing different levels of communication depending on their usage (e.g. video and audio files can be attached to email messages) [16].

2.2. Communication in agile development

Agile methods work with volatile requirements and embrace personal communication between the participants [2]. Since the level of ambiguity can be

considered high in the form of unstable requirements, it seems that personal communication is a natural choice (and in line with MRT) for agile development. Based on the existing literature, Kraut & Streeter [21] summarized the characteristics of informal communication. They state that informal communication is the primary method of exchanging information within research and development organizations [21]. Also in many domains, the physical proximity of the information source is also important, and transmitting the information through informal, interpersonal communication is valuable especially as development and research tasks become more uncertain [21]. Also Cockburn [4] agrees that informal communication between the stakeholders and their physical proximity are important factors. Personal, verbal communication has also been supported e.g. by the following arguments: according to Juric [22] verbal communication enables defect detection early in the development process, and personal contact with the customer causes less distortion to the information [23]. Melnik and Maurer [23] compared the differences between traditional and agile development. Since in traditional development the information travels through a long chain of people, it becomes distorted and some of it is lost. Agile development embraces personal communication with the customer, which reduces both information loss and mutation. [23] Informal and personal communication is thus highly efficient, but it has to be maintained since according to Wake [24], the lack of high-bandwidth communication leads to a loose feedback mechanism. This makes the steering of a project become less responsive, which in turn demands more effort to get the project back on course [24].

Despite its usefulness, verbal communication has also been criticized by e.g. Finsterwalder [5] who has pointed out the pitfalls in verbal communication when discussing Extreme Programming, according to Finsterwalder verbal communication is not always well thought-out and the contents of the conversations can be forgotten during time.

In addition to communication, feedback is another essential success factor (e.g. Wake [24]). The most popular agile method Extreme Programming (XP) embraces communication and feedback as interdependent process values. The *Communication* value of XP requires that the communication must be maximized between the stakeholders through direct, interpersonal communication. *Feedback* value, for its part, requires that the contact with the customer is

close. These values do not exist in isolation but are strongly related; without communication the feedback cannot exist and vice versa [10]. Previously XP defined *On-site customer* practice as requiring the customer's fulltime presence with the team in a same shared workspace [3]. Even though this practice has been found problematic (e.g. [5-8]), it enabled effective communication and instant feedback.

The *On-site customer* has now been replaced by less demanding *Real Customer Involvement* practice which does not explicitly require customer's constant presence. The customer who can contribute to weekly and quarterly planning is still an essential part of the team, and team should have close contact with him [10]. Since the customer is not any longer

required for fulltime presence, the already important roles of communication and feedback become even more essential. Thus it is essential that the communication and feedback mechanisms should receive special attention in agile development.

Gottesdiener [25] has defined a framework for holding requirements workshops with various time and place combinations. This framework can be applied to agile development, thus indicating that the teams can select a proper communication channel from considerably wide collection of different media. In this work, time can be either synchronous with every participant present at the same time or asynchronous. Place can be either co-located or distributed when participants are at different locations.

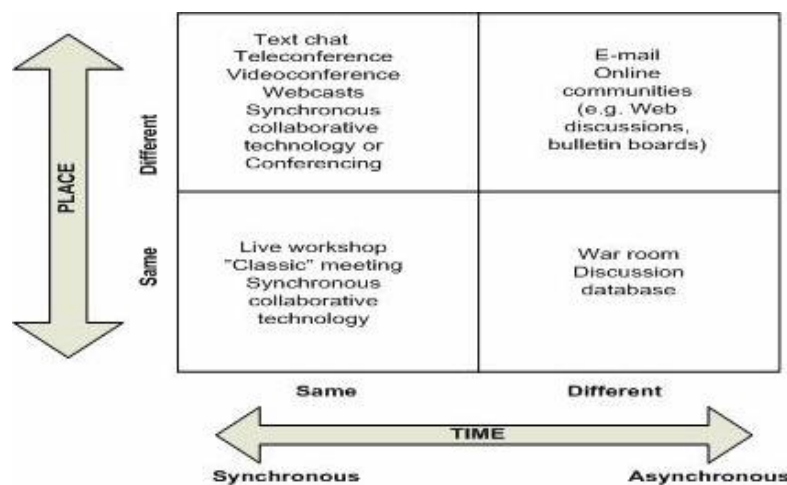


Figure 1. Requirements workshop techniques for different time-place configurations, reproduced from [25].

McLuhan [26] has used the terms "hot" and "cool" when discussing media. McLuhan argues that "cool" media require a higher level of participation from users than "hot" media, and because of the lack of information, much has to be filled in by the users. McLuhan also adds that the use of "hot" and "cold" media has to be adapted to the underlying situation [26]. Ambler and Cockburn [4] have also adapted the use of "hot" and "cold" communication media. They argue that "hot" communication channels provide more information than "cold" media. The following figure is Ambler's [13] modification of Cockburn's comparison of communication effectiveness in different communication media. Different communication channels have been divided into

"cold" and "hot" channels based on their communication effectiveness. "Hot" communication channels are thus more efficient than "cold" media. Both Cockburn's and Ambler's work is based on personal experience and has not been scientifically evaluated.

As can be seen from the graph in Fig. 2, face-to-face communication is the "hottest" communication channel, while email is positioned in the "colder" end of the graph. This division is similar to the classification made by Daft et al in [12]. Thus, both theory and practice indicate that personal communication is the richest form of communication, while less interactive methods are less effective forms of communication.

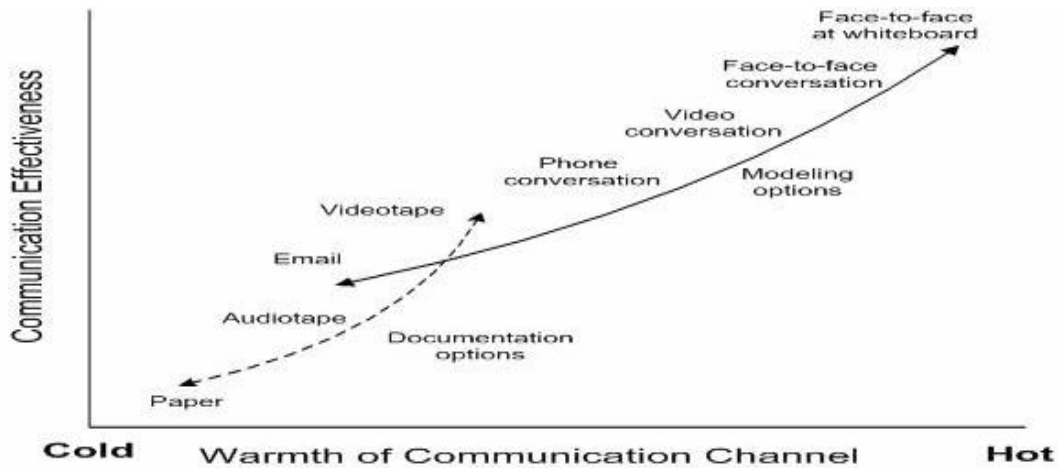


Figure 2. The communication value of different techniques, reproduced from [13].

3. Research design

This section describes the research method, data collection and analysis techniques, and research settings on which the results of this study are based. Our research covered four different case [27] studies, referred to as *Case 1*, *Case 2*, *Case 3* and *Case 4*. *Case 1* has been previously described in [6] and *Case 2* is discussed in [28]. The teams consisted of 5 to 6 developers, and in the case of *Case 3* one of the developers was a member of the customer organization. The aim of projects *Case 2*, *Case 3* and *Case 4* was to implement a mobile software product to the customers, while *Case 1* delivered an application for internal use. In the case of *Case 3*, the assignment was to implement a mobile version of an existing desktop application, implemented by the customer organization.

All the cases were implemented following Mobile-D™ (hereafter Mobile-D). Mobile-D is inspired by several agile methods and it is specifically designed for mobile application development. Mobile-D does

not require a fulltime customer presence. Mobile-D proposes that the customer should participate personally in the development when the contents of the iterations are discussed and analysed, and during the release when the customer can provide feedback about the results of the iterations. Otherwise the customer is not onsite.

Mobile-D consists of 5 iterations. The first and the last iterations last for a week, while the duration of the remaining three is 2 weeks [29]. For the purposes of the study, the development method is only briefly described to provide background information about the research. Mobile-D is explained in greater detail in [29, 30].

Before the projects were launched, the teams (including the customers) were introduced to the philosophy and principles of agile development (Agile Manifesto, available at <http://agilemanifesto.org/>) and trained to use the Mobile-D. The following table summarizes the characteristics of the case projects.

Table 1. Summary of the case projects.

	Case 1	Case 2	Case 3	Case 4
Iterations	5	5	5	5
Iteration span (weeks)	1+2+2+2+1	1+2+2+2+1	1+2+2+2+1	1+2+2+2+1
Number of product releases	4	4	4	4
Product type	Intranet application	Mobile application	Mobile application	Mobile application
Programming	Java	Mobile Java	Mobile Java	Mobile Java

	Case 1	Case 2	Case 3	Case 4
language				
Customer presence	Onsite 80%	Onsite during the beginning and the end of each iteration. Available for face-to-face communication.	Onsite during the beginning and the end of each iteration.	Onsite for the first 2 weeks. Afterwards not onsite.
Customer location	Same facility	Same facility	Same city	Different city
Number of customers	1	3	1	1
Face-to-face communication level	High	On demand	Periodical	Low
Mid-iteration communication media used	Face-to-face	Face-to-face, email	Email	Email and telephone

During our research the utilization of face-to-face communication varied as follows: *High* (onsite customer), *On demand* (face-to-face communication available when needed), *Periodical* (face-to-face communication applied at the beginning and the end of an development iteration) and *Low* (customer onsite for the first two weeks, afterwards face-to-face communication was not applied or available). *Case 1* had an onsite customer, and during *Case 2* the customer was required to be onsite during the Planning Day and Release Day activities of the Mobile-D [29]. These phases concentrate on deciding and analyzing the contents of each development iteration and release, when the results of the iteration are presented to the customer. During these sessions, the customer was available for face-to-face conversation. The mid-iteration communication was managed via email. The customer was, however, available for face-to-face communication on demand, so personal communication and feedback were available also during the iterations. *Case 3* was similar to *Case 2*, excluding the mid-iteration face-to-face communication.

During *Case 4* the customer was constantly available for the first two weeks for guidance and feedback. Thereafter, the communication was managed entirely by email and telephone.

The task data was collected from each case and the defect fixing tasks isolated. The amount of defect-fixing tasks, from all the tasks, were calculated. The teams were required to record the time needed to fix the defects, but this was not properly executed except in the case of *Case 4*. Table 2 summarizes the defect data available from the case projects.

Thus the results from *Case 1*, *Case 2* and *Case 3* can be used only to show trends in the amount of defect densities. It must be noticed that the defect data from the first release of *Case 2* is missing. In addition

to the quantitative research data, qualitative data was also used. This data is based on onsite observations, a research diary and interviews.

Table 2. Available defect data from the case projects.

	Case 1	Case 2	Case 3	Case 4
Defect data	X	X	X	X
Defect fixing times	-	-	-	X

4. Empirical research results

In 4.1 the findings of the study are presented. In 4.2 some techniques for customer communication are discussed based on the existing literature.

4.1. Impact of customer communication on software defects

Both theory and practice have shown that rich face-to-face communication is effective, and its application proved itself very useful in our research. The requirements analysis was conducted face-to-face with the customers and the conversations were further clarified by lightweight paper-prototyping. This approach initiated very lively discussions, thus enabling rich communication between the team and the customers. When compared to the suggestions of Media Richness Theory [11, 12] and personal experiences of both Cockburn [4] and Ambler [13], this finding is not surprising.

In spite of the wide array of different communication methods available for managing mid-iteration communication, only two of the leanest channels described in chapter 2 were used in non-onsite customer cases. *Case 2* is an exception, since

the team was able to discuss with the customer in person even during the development period. Only email (and in *Case 4* also telephone) was used to exchange information between the customers and the rest of the development team during the development. These media were selected, even though the customers were encouraged to choose richer channels. During *Case 2*, the customers specifically asked for simple reports since they were missing the “big picture” from technically oriented reports [28]. The reasons why the customers wanted to use simple reports in the mid-iteration communication during other cases remains unclear. The customers’ inexperience in agile development could be one possible explanation. If the customers are not fully aware of the nature of agile development, the importance of communication and feedback can go unnoticed. During *Case 2*, the customer stated the initial requirements and left the team alone to complete the product without any support or feedback. This situation is described in more detail in [28]. In addition, the findings of Kinney and Watson [18] are interesting. They report that email was found to be an appropriate medium for most communication tasks, including both equivocal tasks, which according to Media Richness Theory should be handled through a richer medium, and non-equivocal tasks. We have some evidence that supports this. When asked about media selection, the customer in *Case 3* responded “*Emails will do just fine, they are enough*”. Thus it seems that the customers were comfortable with simple email reports, and did not want to change the mid-iteration communication medium in spite of the increasing amount of defects.

Even though the customers were pleased with simple reports, which have been found to provide an excellent view of a project’s progress at very low cost [28, 31], it is evident that they do not provide enough information for crucial customer feedback. The following excerpt is from an actual status report sent to the customer in *Case 3*². The report describes the number and name of the stories and their status (in this case either STARTED or DONE) and some brief explanation of their functionalities³.

“*Story 9: Bug Fixes of R2 - DONE*

- *Lots of work with scrolling*

Story 10: Redesign Architecture – STARTED

- *Basically finished, move 1 task to the next release*

Story 11: Orders Are Shown in OrderListScreen - DONE

² Even though the reports did not follow a similar format, the information contents were similar. The reports in all the cases were very simple and described what was done only on a general level.

³ The comments are translated from Finnish.

Story 12: Enhancements to SalesItemScreen - STARTED

- *Week Level still in progress*

Story 13: OrderInfoScreen (Info of One Order) - DONE

Story 14: Search Feature – DONE”

This report indicates that several stories were completed, but it is obvious that the simplicity of the report could not enable the customer to give feedback that could have been used to steer the development. The analysis of the email correspondence between the teams and the customers supports this claim. The customers did not give any feedback based on the reports during *Case 2* and *Case 3*. The customer responded once to an email report explaining implemented features during *Case 4*.

Based on the report above, the customer was not able to verify that the features were in fact implemented correctly, thus making the contents of the report partly ambiguous. On the other hand, ambiguity does not exist when observing stories with the status of STARTED, since these parts are well-defined. Thus this supports the claims of MRT that ambiguous issues should be managed through rich channels, while leaner channels are adequate for well-understood subjects. Similarly, this also supports the works of Cockburn [4] and Ambler [13].

The customers’ inability to give feedback based on the reports further supports the claim by Wake [24], who states that the lack of high bandwidth communication leads to a loose feedback mechanism, which in turn requires more effort to get the project back on course.

The following table summarizes the times in minutes required for defect corrections during *Case 4*. As mentioned in section 3, there is not sufficient data available from other cases. The teams, excluding *Case 4* did not record a defect log in sufficient detail. The defects were at first categorized into customer independent and customer dependent defects based on their descriptions. Customer independent defects are such that can be traced back to developers or to the development platform. Customer dependent defects are defects which could have been avoided by either having constant customer communication and feedback (onsite customer) or utilizing a rich mid-iteration communication method. Based on the defect data analysis, customer independent errors were further divided into two categories. They were either caused by developers or, as in one case, the defect was caused by the underlying Java virtual machine. Similarly the customer dependent defects were divided into cosmetic (e.g. the user interface was not what the customer expected, its elements were not as required,

the customer was not pleased with the colors or fonts) or they had requirement related defects (i.e. the

features did not work as expected).

Table 3. Defect fix times in minutes for customer independent and customer dependent defects in Case 4.

Release	Customer independent		Customer dependent	
#	Developer	Platform	Cosmetic	Requirement
1	310	-	-	-
2	150	-	70	355
3	230	-	160	595
4	30	20	-	30
Subtotal	720	20	230	980
Total	740		1210	

It can be interpreted from the above data that the cost of fixing customer dependent defects was ~1.6 times larger in terms of time when compared to customer independent defects. Further analysis indicated that the percentage of defects in the above table was distributed as follows: *developer*: 37%, *platform*: 1%, *cosmetic*: 12% and *requirements*: 50%. When releases were reviewed separately the amount of customer dependent defects was distributed as set out in Table 4.

Table 4. The proportion of customer dependent defects/release in Case 4.

Release #	% of all defects
1	0
2	73,9
3	76,6
4	37,5

The contents of the first release during all the cases concentrated on the technical infrastructure. In *Case 4* the only implemented feature on which the customer was able to give feedback to the team was login functionality. During this particular iteration, all the defects were caused by the carelessness of the developers.

Clearly, the rate of customer dependent defects was very high during the second and third release and remained considerably high also in the final release. Since face-to-face communication was applied only during the first two weeks, the communication was managed entirely through lean communication channels thereafter. This further supports the MRT's claim that lean channels are not suitable for managing ambiguous issues. In summary, during *Case 4*, 62.6% of software defects

were caused by the lack of efficient customer communication and feedback.

Our other findings, even though not as precise, also support this viewpoint. The following figure shows the percentage of all defect-fixing tasks in each project. The average percentage of defect-fixing tasks was 6.3% in *Case 1* and we can assume that all the defects were in this case developer-oriented. In *Case 2* the percentage of the defect-fixing tasks was reasonably low on average, whereas it increased continuously in *Case 3* and *Case 4*. The last release of *Case 4* concentrated solely on defect-fixing, because all the other scheduled functionalities were cancelled.

For more precise information about the defect task percentages, the following table summarizes the averages for defect-fixing tasks in each case project.

Table 5. The averages for defect-fixing tasks in the case projects.

Project	Utilization of face-to-face communication	Average of defect fixing tasks (%)
<i>Case 1</i>	High	6,3
<i>Case 2</i>	On demand	9,4
<i>Case 3</i>	Periodical	24,7
<i>Case 4</i>	Low	41,8

When combining the figures from *Case 2*, *Case 3* and *Case 4*, 25.3% share of all the tasks were defect fixing tasks in non-onsite customer projects. On the bases of figure 3 and table 5, it can be seen that the averages for defect-fixing tasks in *Case 1* and *Case 2*, and their variation in different releases, are very similar to each other. Even though the customers were not constantly onsite during *Case 2*, and the

mid-iteration communication was managed mainly by email, they were located in the same facility (“He [one of the customers] was almost always there in the room next to us” [Case 2]) and thus easily available for communication and feedback. Thus the amount of face-to-face communication and feedback was higher than in the other non-onsite customer cases. Case 2 had a beneficial situation since the number of customers was higher in Case 2 than in the other cases, and at least one the customers was always available at the beginning and the end of the iterations for face-to-face communication and feedback. Thus the customer communication was regular, while in Case 3 the customer was not always able to join the scheduled meetings which then had to be postponed.

A more alarming trend can be observed when comparing Case 3 and Case 4. These cases applied lower levels of face-to-face communication and

relied more on leaner communication channels, not suitable for ambiguous issues. This backfired in higher defect rates. Case 3 used *Periodical* face-to-face communication and suffered from a 24.7% defect ratio, while Case 4 using a *Low* level of face-to-face communication had a defect rate of 41.8%, with 62.2% of all defects being customer-related. The development of the defect ratios during these cases also follows a similar pattern. The defect rates were constantly increasing as the projects progressed, leaving less time to implement new features or enhance the existing features. Thus our research indicates that the lower adoption of high-bandwidth communication and increasing reliance on lean communication techniques result in higher defect rates.

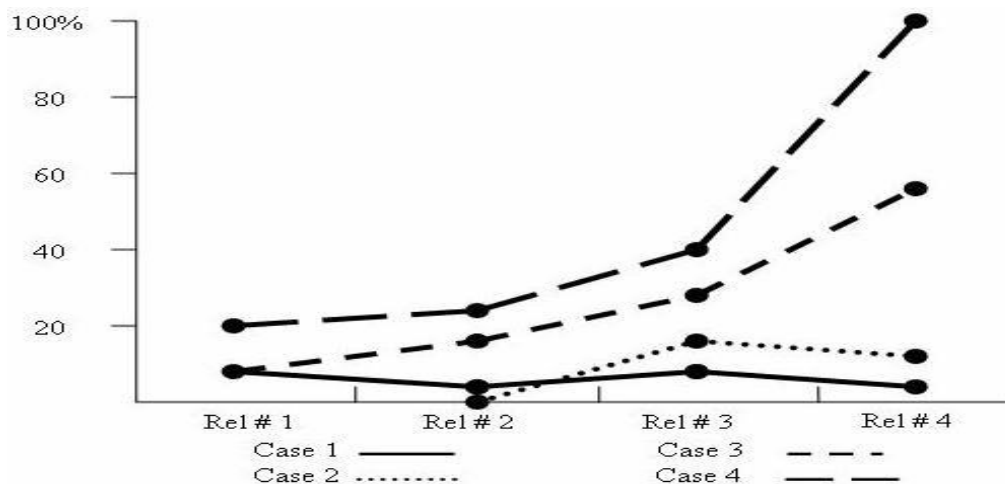


Figure 3. The percentage of defect-fixing tasks in the case projects.

These findings suggest that effective customer communication and selection of efficient communication media, especially during mid-iteration, should receive proper attention in agile development.

4.2. Suggestions for customer communication

As discussed in chapter 2, there is a large amount of different communication mechanisms that can be integrated into agile development as well. The focus of this paper is however out of providing a systematic solution to customer communication in agile development, but based on existing literature

described here and in section 2, some suggestions can be made.

Face-to-face communication is the default communication method in agile development. It has proven itself effective, so it should be applied whenever possible. **Videoconferencing** comes second in richness to face-to-face and e.g. Wallace et al. [32] have suggested its utilization in user story explanation when the customers are remote. We suggest using videoconferencing in situations that need effective customer communication and feedback, where the customer is not available. The usage of videoconferencing has both positive and

negative effects. Sumner and Hostetler [33] have identified the following shortcomings. The difficulty of coordinating and clarifying ideas, and the time required to achieve consensus and arrive at a decision increases [33]. On the other hand, they state that computer conferencing produces better decisions, a wider range of options, greater analysis, and it creates the psychological distance needed to engage in a more open exchange of options. It is also reported in [34] that in the case of requirements negotiation, the teams communicating face-to-face did not outperform teams using computer support.

Telephone: A telephone is not capable of transmitting visual cues, but it enables instant feedback. Telephone can be useful for example for negotiating, for example, schedules with the customer. **Email:** Emails are suitable for communicating well-understood issues.

However, it is not recommended that one blindly chooses a number of different methods for enriching the communication between the customer and the developers. Rather, the decision about communication channels should be agreed in collaboration with the different parties involved, as it also depends on the situation at hand as proposed by Dennis [16] (See also McLuhan [26]).

While the selection of the most appropriate communication method in agile development is not a focus of this paper, we believe it is not a straightforward task and it is a subject that requires further research.

5. Conclusions

Agile software development emphasizes face-to-face communication as the primary communication channel between the project participants. The existing literature has promoted the benefits of face-to-face communication, but less is known about the implications of communication and feedback in the context of agile development.

Our research in four different case studies provides empirical evidence of the effects of customer communication. Despite the considerably wide array of existing communication methods, the customers chose email, and in one case also telephone, to manage both clear and ambiguous issues inside the development iterations. Our research indicates that when the reliance on leaner communication mediums increases, the software defect rates increase accordingly. These results suggest that the selection of communication methods to be used inside development iterations must be paid

significant attention by agile organizations working with partially available customers. Even though it is not the focus of this paper, some suggestions for customer communication are also discussed based on the existing literature.

We claim that a defect follow-up provides a means for increasing our understanding of how well a team has understood a customer's intentions. Yet, we do not encourage developers and customers to aim at reducing defect rates per se. We rather emphasize the importance of intense and meaningful communication. This is bound to decrease the number of misunderstandings, increase the efficiency of the work and offer improved opportunities for learning.

This paper has empirically shown that the defect rates increase due to the use of less informative communications channels in the development time. However, due to the agile principle of short iterations, this is less risky than in traditional software development where the length of an iteration may last for quite long periods. Our empirical data, however, demonstrates that there is room for improvement even in short cyclical agile development by making good use of rich communication mechanisms as we propose in this paper.

6. References

- [1] L. Bernstein. (2005, Taking software requirements creation from folklore to analysis. *ACM SIGSOFT Software Engineering Notes* 30(5), pp. 3-5.
- [2] P. Abrahamsson, O. Salo, J. Ronkainen and J. Warsta, *Agile Software Development Methods: Review and Analysis*. Espoo: VTT Publications, 2002, pp. 107. Available at: www.inf.vtt.fi/pdf/publications/2002/P478.pdf
- [3] K. Beck, *Extreme Programming Explained: Embrace Change*. Upper Saddle River, New Jersey: Addison-Wesley, 2000.
- [4] A. Cockburn, *Agile Software Development*. Indianapolis: Addison-Wesley, 2002.
- [5] M. Finsterwalder, "Does XP need a professional customer", XP2001 workshop on customer involvement, in *XP2001*, Cagliari, Italy, 2001.
- [6] J. Koskela and P. Abrahamsson, "On-site customer in an XP project: Empirical results from a case study," in *EuroSPI2004*, Trondheim, Norway, 2004, pp. 1-11.
- [7] A. Martin, "A case study: Exploring the role of customers on extreme programming projects," Victoria University of Wellington, New Zealand, 2003.

- [8] P. Abrahamsson and T. Jokela, "Usability assessment of an extreme programming project: Close co-operation with the customer does not equal to good usability," in *Profes 2004*, Kansai Science City, Japan, 2004, pp. 393-407.
- [9] R. Jeffries, A. Anderson and C. Hendrickson, *Extreme Programming Installed*. Upper Saddle River, New Jersey, USA: Addison-Wesley, 2001.
- [10] K. Beck and C. Andres, *Extreme Programming Explained; Embrace Change, Second Edition*. Upper Saddle River, NJ, USA: Addison-Wesley, 2005.
- [11] R. L. Daft and R. J. Lengel, "Organizational Information Requirements, Media Richness and Structural Design," *Manage. Sci.*, vol. 32, pp. 554-571, 1986.
- [12] R. L. Daft, R. Lengel and L. K. Trevino, "Message Equivocality, Media Selection, and Manager Performance: Implications for Information Support Systems," *MIS Quarterly*, vol. 11, pp. 355-366, 1987.
- [13] S. W. Ambler, "Validating Agile Models," *Cutter IT Journal*, vol. 15, pp. 33-39, 2002.
- [14] A. Edstrom, "User Influence and the Success of MIS Projects: A Contingency Approach," *Human Relations*, vol. 30, pp. 580-607, 1997.
- [15] R. B. Bostrom and B. D. Thomas, "Achieving excellence in communications: A key to developing complete, accurate and shared information requirements," in *Proceedings of the Twentieth Annual Computer Personnel on Research Conference*, Charlottesville, Virginia, United States, 1983, pp. 1-13.
- [16] A. R. Dennis and J. S. Valacich, "Rethinking media richness: Towards a theory of media synchronicity," in *HICSS'99*, Hawaii, United States, 1999, pp. 1017.
- [17] S. Kinney and A. Dennis, "Reevaluating media richness: Cues, feedback, and task," in *HICSS'94*, Hawaii, United States, 1994, pp. 21-30.
- [18] S. T. Kinney and R. T. Watson, "The Effect of Medium and Task on Dyadic Communication" in *Proceedings of the 13th International Conference on Information Systems*, Dallas, Texas, United States, 1992, pp. 107-117.
- [19] R. L. Daft and K. Weick, "Toward a Model of Organizations as Interpretation Systems," *Academy of Management Review*, vol. 9, pp. 284-295, 1984.
- [20] A. Graveline, C. Geisler and M. Danchak, "Teaming together apart: Emergent patterns of media use in collaboration at a distance," in *Proceedings of the 2000 joint IEEE International and 18th Annual Conference on Computer Documentation*, Cambridge, Massachusetts, United States, 2000, pp. 381-393.
- [21] R. E. Kraut and L. A. Streeter, "Coordination in Software Development," *Communications of the ACM*, vol. 38, pp. 69-81, 1995.
- [22] R. Juric. "Extreme programming and its development practices". in *Proceedings of the 22nd International Conference on Information Technology Interfaces, ITI2000*, Pula, Croatia, 2000, pp. 97-104.
- [23] G. Mehnik and F. Maurer, "Direct verbal communication as a catalyst of agile knowledge sharing," in *Agile2004*, Salt Lake City, Utah, United States, 2004, pp. 21-31.
- [24] W. C. Wake, *Extreme Programming Explored*. Addison-Wesley, 2002.
- [25] E. Gottesdiener, *Requirements by Collaboration: Workshops for defining needs*. Boston: Addison-Wesley, 2002.
- [26] M. McLuhan, *Understanding Media: The Extensions of Man*. New York: McGraw Hill, 1964.
- [27] R. K. Yin, *Case Study Research Design and Methods*. 2nd ed. Sage Publications, 1994.
- [28] M. Korkala and P. Abrahamsson, "Extreme programming: Reassessing the requirements management process for an offsite customer," in *EuroSPI 2004*, Trondheim, Norway, 2004, pp. 12-22.
- [29] T. Ihme and P. Abrahamsson, "Agile Architecting: The Use of Architectural Patterns in Mobile Java Applications," *IJAM*, 2005.
- [30] P. Abrahamsson, A. Hanhineva, H. Hulkko, T. Ihme, J. Jäälinoja, M. Korkala, J. Koskela, P. Kyllönen and O. Salo, "Mobile-D: An agile approach for mobile application development," in *OOPSLA 2004*, Poster session, Vancouver, Canada, 2004.
- [31] M. Korkala, "Extreme programming: Introducing a requirements management process for an offsite customer," Department of Information Processing Science Research Papers Series A, University of Oulu, Finland, 2004.
- [32] N. Wallace, P. Bailey and N. Ashworth, "Managing XP with multiple or remote customers," in *3rd International Conference on eXtreme Programming and Agile Processes in Software Engineering (XP2002)*, Alghero, Italy, 2002, pp. 134-137.
- [33] M. Sumner and D. Hostetler, "A comparative study of computer conferencing and face-to-face communications in systems design," in *2000 ACM SIGCPR Conference on Computer Personnel Research*, Chicago, Illinois, United States, 2000, pp. 93-99.
- [34] D. E. H. Damian, A. Eberlein, M. L. G. Shaw and B. R. Gaines, "Using Different Communication Media in Requirements Negotiation," *IEEE Software*, vol. 17, pp. 28-36, 2000.